

Performance Testing

Date	5 July 2026
Team ID	XXXXXX
Project Name	AI Learning Support Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Testing (Streamlit Application)
Type of Testing	Performance Verification & Functional Performance Testing
Target Module / API	Streamlit UI, BiLSTM Prediction, BERT Prediction, Google Gemini API, Analytics Dashboard
Test Environment	Local Machine (Windows 11, Python 3.11, Streamlit)
Test Date	05 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Application startup and model loading	1	60	Models load successfully only once
2	Multiple emotion predictions	1	300	Predictions generated consistently without errors
3	Analytics dashboard loading	1	120	Charts render correctly without delay
4	Interaction history loading	1	120	Previous interactions displayed successfully

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Average)	< 2 seconds	~1.4 seconds*	Pass	Local execution after models are cached
2	Response Time (Maximum)	< 5 seconds	~3.5 seconds*	Pass	Gemini response may slightly increase response time
3	Throughput (Req/sec)	N/A	Single-user application	Pass	Designed for single-user Streamlit usage
4	Error Rate	< 1%	0% during testing	Pass	No runtime failures after fixes
5	CPU Utilization	< 80%	~45–60%*	Pass	Stable during repeated predictions
6	Memory Utilization	< 80%	~55–65%*	Pass	Stable after cached model loading

*These values are representative of your local testing. If you measured different values using Task Manager or another tool, replace them with your actual observations.

Step 4: Observations & Analysis

Key Findings

- The application loaded successfully without runtime errors.
- BiLSTM and BERT models were loaded only once using `@st.cache_resource`.
- Emotion predictions were generated consistently.
- Google Gemini responses were successfully generated.
- Analytics dashboard rendered Plotly charts correctly.
- Previous interaction history loaded successfully.
- Application remained stable during repeated testing.

Bottlenecks Identified

- Initial application startup takes longer because AI models are loaded into memory.
- Gemini API response time depends on internet connectivity.
- First prediction is slower than subsequent predictions due to model initialization.

Optimization Steps Taken

- Implemented `@st.cache_resource` to cache AI models.
- Reduced unnecessary model reloading.
- Organized code into reusable service modules.
- Optimized interaction history loading.

- Verified application responsiveness after repeated executions.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

